

PROCEDURAL MEMORY FOR CODING AGENTS

DejaDo

A new kind of memory for coding agents. Graph-based, shared, earned from experience.

9,000

VM sessions relearning the same
repo, the same discovery, paid nine
thousand times.

Agents share no context between
sessions.

Every session starts from zero.

They learn from manuals, not
experience.

AGENTS.md and docs are written once, by hand.

A universal memory layer under every agent, built the way memory is built: from experience, not instructions.

01 RECORD

The trace

```
.headstart/sessions/<id>.  
jsonl
```

Every tool call: read, search, write, execute. Files touched, commands run, exit code.

src/hook.ts, src/trace.ts

02 CONSOLIDATE

The procedure

```
trace → procedure  
no model call
```

Deterministic extraction: steps, files_written, preconditions, postconditions.

src/extract.ts

03 RETRIEVE

By similarity

```
ncd(a,b) = (C(ab) -  
min(C(a),C(b))) /  
max(C(a),C(b))
```

gzip level 9, in bytes. How much of one trace explains another. No embedding, no model.

src/ncd.ts, src/recall.ts

04 STRUCTURE

Hierarchy and graph

```
cutAt(0.50) → communities  
cutAt(0.62) → shapes
```

Steps become a state and action graph. Longer and new workflows inferred from shared nodes.

src/graph.ts

A procedure is a path. Two procedures that ran the same step meet at that node.

States it needed, files it touched, commands it ran, what it verified, drawn as one map of three real procedures on the same fixture.



Nearest neighbour by this distance lands on the same task shape **90 out of 90** times, the same task **88 out of 90**.

A Devin procedure, handed to Claude Code, and back.

Three waves of two, one plan file, no prompt shared between the two agents. Measured on Sep 19.

LANE	AGENT	TASK	CALLS	BEFORE	EDIT	TIME	HANDED
devin-1	devin	bugfix-1	12		5	40s	cold
claude-1	claude	endpoint-1	19		9	51s	cold
claude-2	claude	bugfix-1	5		2	25s	handed 1 (devin bugfix-1)
devin-2	devin	endpoint-1	13		8	36s	handed 1 (claude endpoint-1)
devin-3	devin	bugfix-2	12		8	54s	handed 1 (devin bugfix-1)
claude-3	claude	endpoint-2	27		12	112s	handed 2 (claude endpoint-1, devin bugfix-1)

12 → 5

devin bugfix-1, cold, to claude, handed

19 → 13

claude endpoint-1, cold, to devin, handed

Same fixture, same model, cold vs with memory.

Every number is a recorded session.

Claude Code

n = 18 per arm

Tool calls	18.2 → 14.8	-18%
Context tokens	1,009,927 → 758,606	-25%
Seconds	75.1 → 59.4	-21%
Cost per run	\$0.423 → \$0.366	-13%

Devin CLI

n = 18 per arm

Tool calls	17.7 → 16.5	-7%
Context tokens	593,882 → 462,350	-22%
Seconds	64.5 → 54.3	-16%
Cost per run	not reported by runner	

One CLI, two endpoints, any agent that pipes JSON.

headstart install

write hook files for Claude Code, Devin CLI, Codex

headstart hook <Event>

stdin: one hook event in, JSON out

headstart recall "task"

the best procedure for a task

headstart list

list stored procedures

headstart skills

write SKILL.md files Devin, Warp and Codex discover on their own

headstart run

one real agent session, traced into a lane

headstart demo

before and after, one command

headstart orchestrate

waves of runs, hand-offs across agents

headstart push

backfill a session into the hosted store

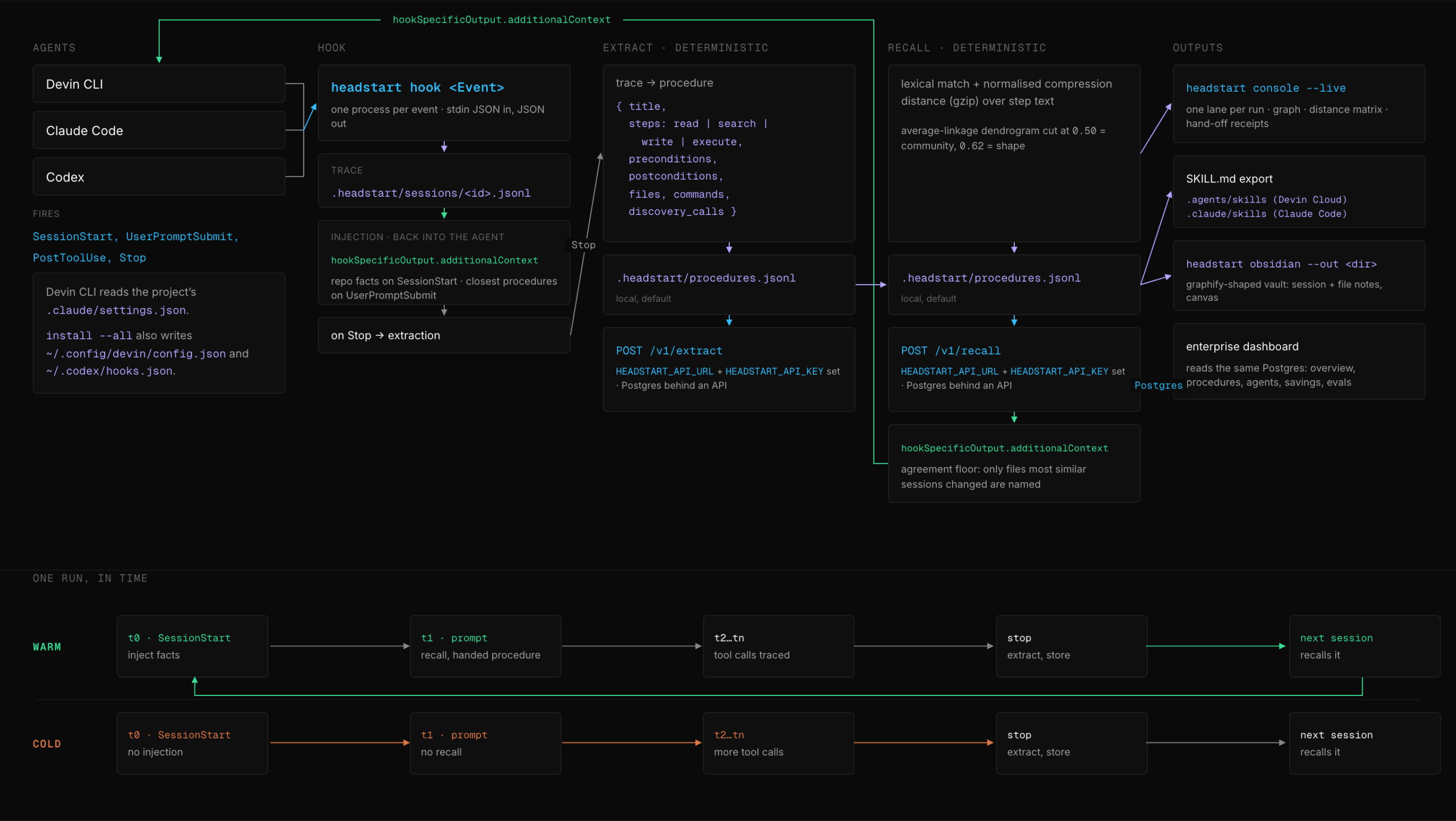
POST /v1/extract

a finished session becomes a stored procedure

POST /v1/recall

the closest stored procedures to a prompt

Works under Claude Code, Devin CLI and Codex today. Any agent that can pipe JSON on stdin.



Deterministic: extract · trace to procedure, no model call. recall · lexical match plus gzip compression distance, no model call.

Not deterministic: the agent · what it reads, runs, and decides to write each session.

DEJADO

The second session starts where the first one finished.

Repo

github.com/AdvaiytSane/environment-mem

Demo

headstart-demo.vercel.app/dash/enterprise